DAILY PLANNER

ADVANCED DATABASE - Assignment

Submitted By:

Kailash Badu

Roll Number: - 09

BSC. CSIT

Eight Semester

Symbol Number: 28986/078

Submitted To:

Department of TU

R.D. Bhatta

DAILY PLANNER

1 TABLE OF CONTENTS

Table of Contents

1 TABLE OF CONTENTS 1

2 EXECUTIVE SUMMARY 4

2.1 Objective of the report 4

3 INTRODUCTION 5

3.1 Why MongoDB? 5

3.1.1 Document-Oriented Storage 5

3.1.2 Schema-Less by Nature 6

3.1.3 Non-Relational Data Model — No JOINs Needed 6

3.1.4 Flexible Schema Evolution 6

3.1.5 Native MongoDB Features Applied in This Project 6

4 SYSTEM DESIGN & IMPLEMENTATION 7

4.1 Technology Stack 7

4.2 Project Structure & Architecture 7

4.2.1 config/ 7

4.2.2 models/ 7

4.2.3 controllers/ 8

4.2.4 routes/ 8

4.2.5 middlewares/ 8

4.2.6 utils/ 8

4.3 UML Diagrams (Unified Modeling Language) 8

4.3.1 Use Case Diagram 9

4.3.2 Class Diagram 10

4.3.3 Sequence Diagram 11

4.3.4 Entity Relationship (ER) Diagram 12

4.4 MongoDB — Database Design Deep Dive 13
4.4.1 Database & Collection Overview 13
4.4.2 Schema Design Decisions 13
4.4.3 Indexes 14
4.4.4 ObjectId References (Manual References Pattern) 14
4.4.5 Connection Management 15
4.4.6 Environment-Based Configuration 15
4.5 Data Models 15
4.5.1 User Model — USERS Collection 15
4.5.2 Todo Model — TODOS Collection 16
4.5.3 Refresh Token Model — REFRESH_TOKENS Collection 16
4.6 Authentication System 16
4.6.1 Registration (POST /auth/register) 16
4.6.2 Login (POST /auth/login) 17
4.6.3 Token Refresh (POST /auth/refresh) 17
4.6.4 Logout (POST /auth/logout) 17
4.6.5 Current User (GET /auth/me) 17
4.6.6 Cookie & JWT Security Configuration 17
4.7 API Endpoints 17
4.7.1 Authentication Routes — Prefix: /auth 17
4.7.2 Todo Routes — Prefix: /todos (All Require Authentication) 18
4.7.3 Request & Response Examples 18
4.8 Middleware Pipeline & Error Handling 20
4.8.1 catchAsync & AppError 20
4.8.2 Error Response Shape 20
5 CONCLUSION 20
6 RECOMMENDATIONS 21
6.1 Input Validation with Zod 21
6.2 Refresh Token Rotation 21

6.3 Rate Limiting on Auth Endpoints 21
6.4 MongoDB Compound Index on TODOS 21
6.5 Automated Testing with Jest and Supertest 22
6.6 Remove Default Secret Fallbacks 22
6.7 CORS Configuration 22
7 User Interface & Application Screenshots 22
8 References 25

Table of Figures

Figure 4.3.1 - Use Case Diagram: Daily Planner API 9
Figure 4.3.2 - Class Diagram: Key interfaces, classes, and modules 10
Figure 4.3.3 - (a) Sequence Diagram: User Login Flow 11
Figure 4.3.4 - ER Diagram: MongoDB Collections and Relationships 12
Figure 6.7.1 - Login Page 22
Figure 6.7.2 - Sign Up Page 23
Figure 6.7.3 - Dashboard 23
Figure 6.7.4 - Create and Get Todo 24
Figure 6.7.5 - Update Todo 24

Table of Tables

Table 4.4-1: Index Table 14
Table 4.5-1: users collection 15
Table 4.5-2: Todos collection 16
Table 4.5-3: Refresh tokens collections 16
Table 4.7-1: Authentication Routes 18
Table 4.7-2: Todo Routes 18

2 EXECUTIVE SUMMARY

The Daily Planner is a backend web application built with Node.js, Express, TypeScript, and MongoDB. It exposes a secure RESTful API for managing personal to-do items alongside a fully featured authentication system using JWT access and refresh tokens delivered via HTTP-only cookies.

The codebase is structured in a clean, layered architecture, separating configuration, models, controllers, routes, middleware, and utilities — making it maintainable, readable, and extensible. Errors are handled uniformly through a custom AppError class and a global error-handling middleware, and all asynchronous handlers are wrapped with a catchAsync utility to eliminate repetitive try/catch blocks.

The project uses MongoDB as its primary data store, with three collections: USERS, TODOS, and REFRESH_TOKENS. Schema design, indexing, and a TTL-based token cleanup strategy reflect production-grade MongoDB practices.

2.1 Objective of the report

The primary objective of this project was to design and build a fully functional task management REST API while gaining hands-on experience with MongoDB — a leading NoSQL, schema-less, non-relational database. A core learning goal was to understand how MongoDB differs from traditional relational databases (such as MySQL or PostgreSQL) and to apply its unique features — document storage, dynamic schemas, embedded references, and native indexing mechanisms like TTL indexes — in a real-world application context.

A secondary objective was to integrate MongoDB with a modern TypeScript/Node.js backend using the Mongoose ODM, learning how to define schemas, enforce validation, and manage relationships between collections without the constraints of a rigid relational model. Through this project, the following specific skills and concepts were developed and demonstrated:

- Understanding MongoDB's document-oriented, schema-less nature and how it enables flexible, iterative data modelling.
- Designing three MongoDB collections — USERS, TODOS, and REFRESH_TOKENS — each with intentional schema decisions suited to the non-relational paradigm.
- Applying MongoDB-native features: unique indexes, TTL (Time-To-Live) indexes for automatic document expiry, and ObjectId-based manual references between collections.
- Using Mongoose to bridge the schema-less flexibility of MongoDB with the type safety of TypeScript — defining schemas, validation rules, default values, and lifecycle hooks (pre-save middleware).

- Appreciating the advantages of a non-relational database for this use case: no JOIN operations needed, documents are self-contained, and schema evolution requires no migration scripts.
- Presenting UML diagrams — Use Case, Class, Sequence, and ER — to document system design.
- Implementing a secure JWT authentication system and a clean, layered Express.js API architecture.

3 INTRODUCTION

Task management applications are among the most practically useful tools a developer can build, as they involve every core concern of a real-world backend: user identity, data ownership, access control, and full CRUD operations. The Daily Planner project was undertaken as part of a college curriculum to put these concepts into practice using an industry-relevant technology stack.

The backend is written entirely in TypeScript, which adds compile-time type safety on top of JavaScript. Express.js was chosen for its minimalism and flexibility, and MongoDB, accessed via the Mongoose ODM, provides a schema-based document store that maps naturally to the JSON responses the API produces.

Authentication is a central feature. Rather than a simple login that returns a single token, the system uses a dual-token strategy: a short-lived JWT access token for authorizing API requests, and a long-lived refresh token persisted in MongoDB for session continuation. Both are delivered as HTTP-only cookies, mirroring the security posture of production-grade applications.

The project repository is organized under 404bad/college/daily_planner on GitHub, with source code in src/ and a compiled output served alongside a static HTML frontend in public.

3.1 Why MongoDB?

One of the explicit learning goals of this project was to gain hands-on experience with MongoDB and to understand what makes a non-relational, document-oriented database fundamentally different from traditional relational databases like MySQL or PostgreSQL. The following points highlight the key MongoDB concepts explored and applied throughout this project:

3.1.1 Document-Oriented Storage

MongoDB stores data as BSON (Binary JSON) documents, self-contained records that can hold nested objects and arrays. Unlike a relational table where every row must conform to a fixed set of columns, a MongoDB document holds exactly the fields it needs [1]. In the Daily Planner, a todo document stores all of its data, title, description, dates, completion status, and the owner reference , in a single document readable in one query, with no JOIN across multiple tables required.

3.1.2 Schema-Less by Nature

MongoDB imposes no fixed schema at the database level. Two documents in the same collection can have entirely different fields. This makes MongoDB extremely flexible during development — adding a new field to a user document requires no ALTER TABLE statement, no migration script, and no downtime [1].

In this project, Mongoose is used on top of MongoDB to define voluntary schemas in application code [2] (user.model.ts, todo.model.ts, refreshToken.model.ts). This provides the benefits of schema-less flexibility at the database level while still enforcing structure and validation within the Node.js application , the best of both worlds.

3.1.3 Non-Relational Data Model — No JOINs Needed

Relational databases model data across multiple tables and use JOIN operations to combine them at query time. MongoDB takes a different approach: related data can either be embedded inside a parent document or referenced by storing an ObjectId [1]. In the Daily Planner, todos reference their owner via a user_id ObjectId field. Finding all todos for a user is a simple find({ user_id: userId }) query on a single collection — no cross-table join required.

3.1.4 Flexible Schema Evolution

Because MongoDB does not enforce a rigid column structure, the data model can evolve freely as requirements change. If a future version needed to add a priority field or a tags array to todos, those fields could be added to new documents immediately. Existing documents without those fields remain valid, MongoDB simply returns undefined for the missing field rather than throwing a schema violation error. In a relational database, this would require a migration to ALTER the table and backfill all existing rows [1].

3.1.5 Native MongoDB Features Applied in This Project

- TTL (Time-To-Live) Indexes, The REFRESH_TOKENS collection uses a TTL index on expiresAt. MongoDB automatically deletes expired token documents with no cron job or manual cleanup needed. This is a MongoDB-native feature with no direct SQL equivalent [3].
- Unique Indexes, The username and email fields in USERS, and the token field in REFRESH_TOKENS, carry unique indexes enforced at the storage engine level, rejecting duplicates before they reach application code [3].
- ObjectId — Every document is assigned a globally unique _id of type ObjectId by default. ObjectIds encode a timestamp, machine identifier, and counter, making them sortable by insertion time and safe to generate across distributed systems [3].
- Mongoose Pre-Save Hooks, The User schema uses a pre('save') hook to auto-generate a DiceBear avatar URL before the document is written to MongoDB, keeping controller logic clean [3].

- select: false — Mongoose's select: false on the password field instructs MongoDB to exclude it from every standard query result, preventing accidental password exposure anywhere in the application [3].

4 SYSTEM DESIGN & IMPLEMENTATION

4.1 Technology Stack

The following technologies form the foundation of the Daily Planner application:

- Node.js:- Server-side JavaScript runtime. Powers the Express HTTP server and all I/O operations [4].
- Express.js:- Minimal web framework providing routing, middleware chaining, and request/response handling [5].
- TypeScript:- Statically typed superset of JavaScript. Provides interfaces, type checking, and safer refactoring across the entire codebase [6].
- MongoDB:- NoSQL document database. Stores application data across three collections: USERS, TODOS, and REFRESH_TOKENS.
- Mongoose:- ODM library adding schema definitions, validation, middleware hooks, and query helpers on top of the MongoDB Node.js driver [2].
- jsonwebtoken :- Signs and verifies JWTs used as access and refresh credentials [7].
- Bcryptjs:- Hashes and compares passwords using the bcrypt adaptive hashing algorithm [8].
- cookie-parser:- Parses HTTP cookies from incoming request headers into req.cookies [9].
- dotenv:- Loads .env file variables into process.env at startup [10].

4.2 Project Structure & Architecture

The application follows a layered MVC-inspired architecture. All source code lives in src/ and is divided into focused subdirectories:

4.2.1 config/

env.config.ts reads all environment variables through dotenv and exports them as a strongly typed config object with safe development defaults. db.config.ts exposes connectDB() and disconnectDB(), which manage the Mongoose connection lifecycle with error handling that terminates the process on startup failure.

4.2.2 models/

Three Mongoose schemas map to three MongoDB collections. user.model.ts defines the USERS collection with a pre-save hook for auto-generating avatars. todo.model.ts defines the TODOS collection with timestamps enabled. refreshToken.model.ts defines REFRESH_TOKENS with a TTL index for automatic document expiry.

4.2.3 controllers/

Controllers handle incoming requests, extract data from req.body or req.params, delegate to service functions, and return JSON responses. auth.controller.ts covers register, login, logout, refresh, and /me. todo.controller.ts covers the four CRUD operations on todo items.

4.2.4 routes/

auth.route.ts registers the five authentication endpoints under the /auth prefix. todo.route.ts applies authMiddleware globally using router.use(), so every todo route automatically requires a valid access token without repeating the guard on each handler.

4.2.5 middlewares/

auth.middleware.ts verifies the JWT access token (from Bearer header or cookie) and injects the decoded userId into the request as the AuthRequest interface. errorHandler.middleware.ts is registered last and formats all errors — AppError instances and generic Errors alike — into a consistent JSON response structure.

4.2.6 utils/

- **AppError.ts** — Custom Error subclass with statusCode and status fields. Fixes the prototype chain for correct instanceof checks.
- **catchAsync.util.ts** — Higher-order function wrapping async handlers to forward rejected promises to next() automatically.
- jwt.util.ts — generateAccessToken() signs a payload of { userId, username } using the configured secret and expiration.
- password.util.ts — signupPasswordMatcher(), hashPassword() (bcrypt), and verifyPassword() (bcrypt compare).
- cookie.util.ts — Four functions to set/clear access and refresh token cookies with httpOnly, secure, and sameSite flags.
- sanitizeUser.util.ts — Converts a Mongoose document to plain object and removes the password field before any response.

4.3 UML Diagrams (Unified Modeling Language)

The following four UML diagrams describe the structure and behaviour of the Daily Planner system from different perspectives — who uses it, how its data is shaped, how its code is organised, and how its key flows execute at runtime.

4.3.1 Use Case Diagram

The Use Case Diagram shows the interactions between external actors and the system. The two actors are an Unauthenticated User (a visitor who has no session) and an Authenticated User (a visitor who holds a valid access token cookie). The system boundary is the Daily Planner API.

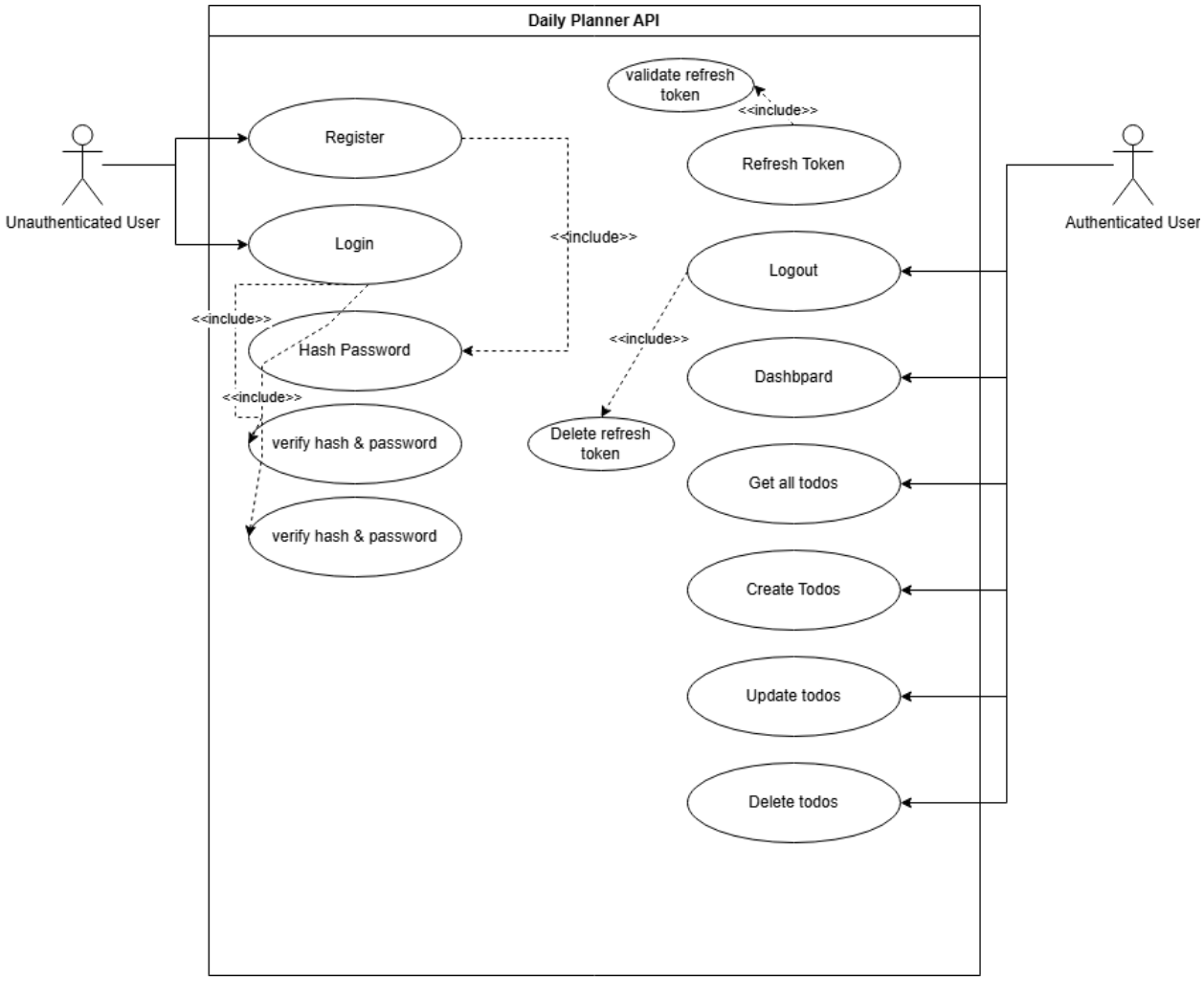

Figure 4.3.1 - Use Case Diagram: Daily Planner API

The Register and Login use cases are available to unauthenticated users. Both include sub-cases: Register includes hashing the password with bcrypt and issuing access + refresh token cookies; Login includes verifying the bcrypt hash and then also issuing tokens. All remaining use cases — Refresh Token, Logout, View My Profile, and all four Todo operations — are only accessible to an authenticated user whose request carries a valid access token cookie verified by authMiddleware. Create Todo additionally includes a field validation step (title and due_date are required). Update Todo and Delete Todo are further scoped to items owned by the requesting user, enforced by the service layer querying with both the todo _id and the req.userId.

4.3.2 Class Diagram

The Class Diagram shows the key TypeScript interfaces and classes in the system, their fields, methods, and the relationships between them. Note that because this project uses Mongoose (a schema-based ODM), 'classes' correspond to Mongoose Document interfaces and the modules that operate on them.

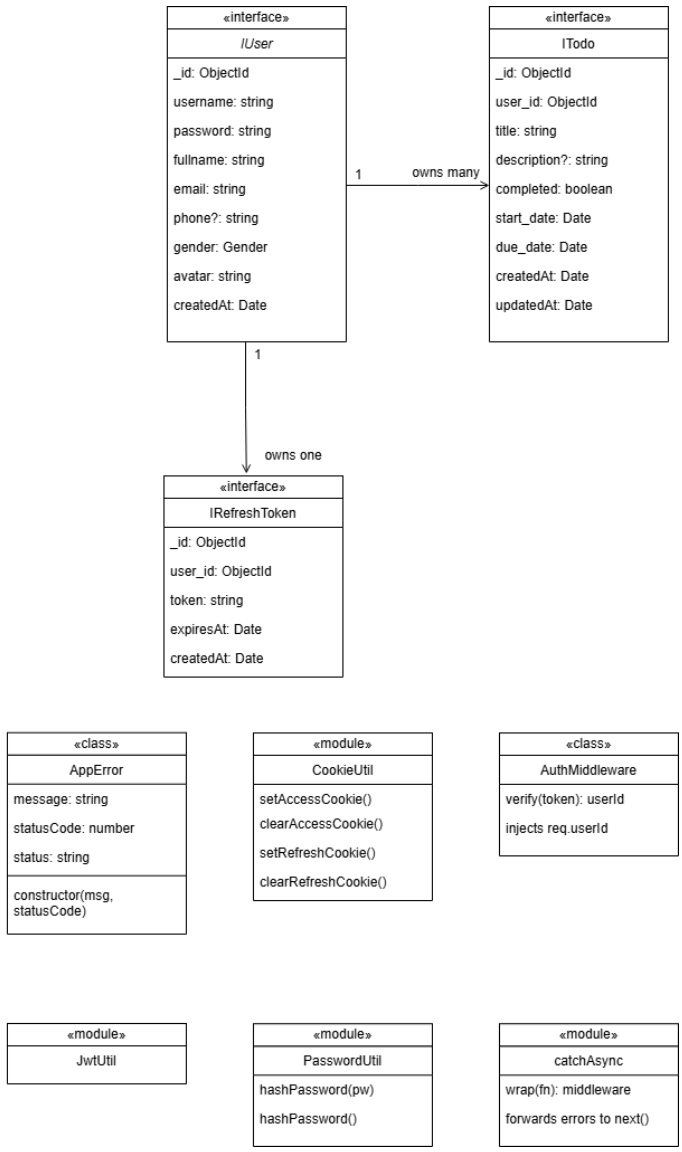

Figure 4.3.2 - Class Diagram: Key interfaces, classes, and modules

IUser, ITodo, and IRefreshToken are TypeScript interfaces that extend Mongoose's Document type. IUser has a one-to-many relationship with ITodo — one user can own many todos, enforced via the user_id ObjectId reference field on every todo document. IUser also has a one-to-one relationship with IRefreshToken — at most one active refresh token is stored per user at any time (the getOrCreateRefreshToken service function upserts rather than creates a duplicate). The utility

modules — JwtUtil, PasswordUtil, CookieUtil, catchAsync, and AppError — are stateless and interact with the model interfaces indirectly through the controller and service layers.

4.3.3 Sequence Diagram

The Sequence Diagram traces the complete message flow for two of the most important operations in the system: User Login and Create Todo. These two flows together cover every major layer of the application — routing, middleware, controller, service, utility, and database.

4.3.3.1 Sequence A — User Login Flow

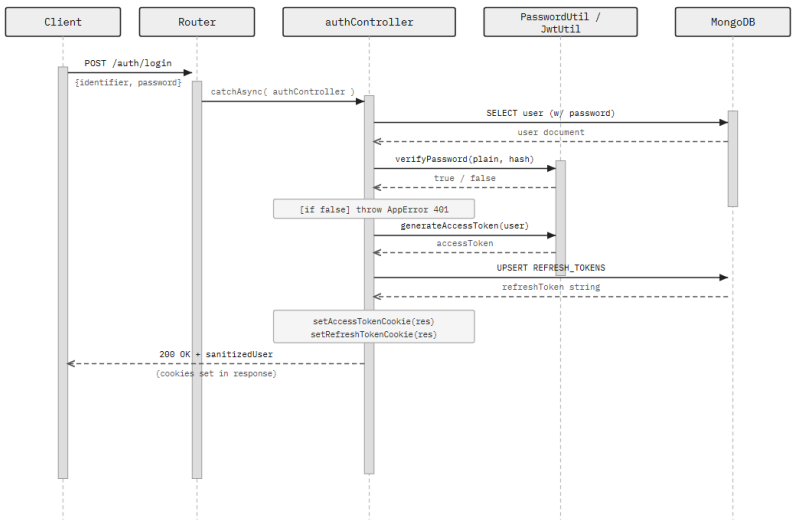

Figure 4.3.3 - (a) Sequence Diagram: User Login Flow

4.3.3.2 Sequence B — Create Todo Flow (Authenticated)

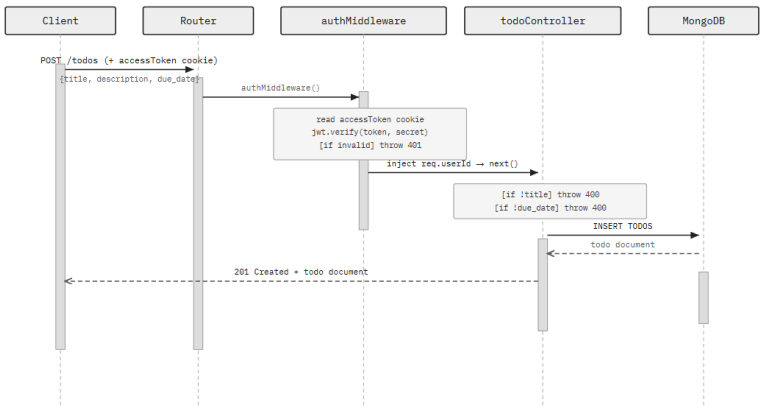

Figure 4.3.3 - (b) Sequence Diagram: Create Todo Flow

Sequence A illustrates that the login controller never short-circuits after a 'user not found' result — it always proceeds to the password verification step using bcrypt.compare(), returning the same

generic 'Invalid credentials.' error regardless of whether the user was absent or the password was wrong. This prevents an attacker from distinguishing between the two failure modes (username enumeration prevention).

Sequence B shows how authMiddleware intercepts the request before it reaches the controller. The middleware reads the accessToken cookie, calls jwt.verify(), and either injects req.userId and calls next(), or throws a 401 AppError that bypasses the controller entirely and is caught by catchAsync, then forwarded to the global errorHandler.

4.3.4 Entity Relationship (ER) Diagram

The ER Diagram shows the three MongoDB collections in the Daily Planner database — USERS, TODOS, and REFRESH_TOKENS — their fields, data types, and the relationships between them. Because MongoDB is a document database rather than a relational one, 'relationships' are represented via ObjectId reference fields rather than foreign key constraints. Mongoose's ref option enables populate() queries across collections.

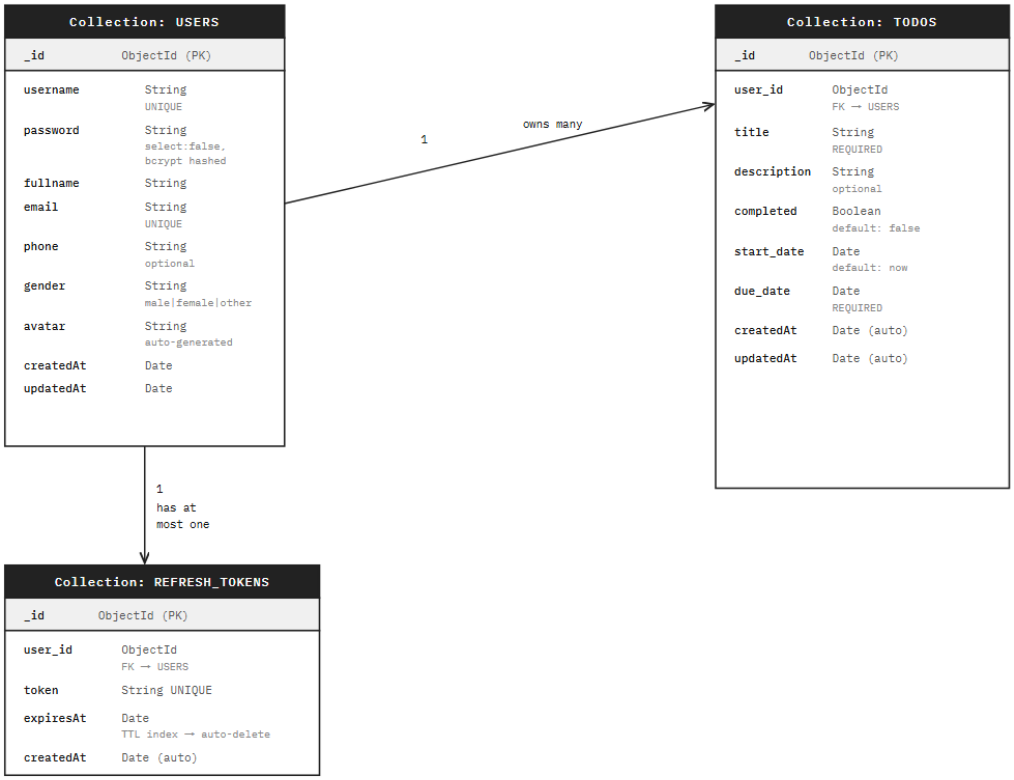

Figure 4.3.4 - ER Diagram: MongoDB Collections and Relationships

The USERS collection is the central entity. Each user document has a one-to-many relationship with the TODOS collection — a single user can own any number of todos, each of which stores the owner's _id in its user_id field. The service layer always queries todos using both the document

_id and the req.userId, ensuring users can only read, update, or delete their own items. USERS also has a one-to-one (at most one) relationship with REFRESH_TOKENS — the getOrCreateRefreshToken() service upserts a single token per user rather than accumulating multiple active sessions.

4.4 MongoDB — Database Design Deep Dive

The Daily Planner uses MongoDB as its sole data store. MongoDB is a document-oriented NoSQL database that stores records as BSON (Binary JSON) documents within collections. It was chosen for this project because the data structures — users, tasks, and tokens — are naturally document-shaped, schema evolution is straightforward during development, and Mongoose provides an ergonomic layer for schema definition, validation, and middleware hooks in a Node.js environment.

4.4.1 Database & Collection Overview

The application uses a single MongoDB database (identified by the connection string in MONGO_URI) containing three collections:

- **USERS**: Stores user account documents. One document per registered user.
- **TODOS**: Stores task documents. One document per todo item, referencing its owner by user_id.
- **REFRESH_TOKENS**: Stores active refresh token documents. At most one document per user at any time.

4.4.2 Schema Design Decisions

Each schema was designed with specific MongoDB considerations in mind:

USERS collection: The password field is defined with select: false in the Mongoose schema. This means that every standard find() or findById() query automatically omits the password from the returned document. Only queries that explicitly use .select('+password') will include it — which is done only in the login flow via findUserByIdentifierWithPassword(). This prevents accidental password leakage in any other query across the application. The username and email fields carry unique: true, which causes Mongoose to instruct MongoDB to create unique indexes on these fields automatically.

A pre-save hook on the User schema auto-generates an avatar URL using the DiceBear Notionists API (seeded with the username) if no avatar is provided. This hook runs only when this.avatar is falsy, ensuring it does not overwrite a user-supplied avatar on subsequent saves.

TODOS collection: Timestamps are enabled via { timestamps: true } in the schema options. This instructs Mongoose to automatically manage two Date fields: createdAt (set on first insert) and updatedAt (updated on every save). The user_id field is typed as

mongoose.Schema.Types.ObjectId with ref: 'USERS', enabling Mongoose's populate() for joining user data in future queries. The completed field defaults to false, meaning every newly created todo starts as incomplete without the client needing to supply this field.

REFRESH_TOKENS collection: The most architecturally significant schema. The token field is unique — preventing duplicate token documents. The critical feature is the TTL (Time-To-Live) index: refreshTokenSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });

This instructs MongoDB to automatically delete any document in this collection once its expiresAt timestamp is in the past. The expireAfterSeconds: 0 setting means MongoDB's background TTL monitor deletes documents at the exact expiry time (within approximately 60 seconds, the monitor's polling interval). This eliminates the need for any scheduled job, cron task, or manual cleanup logic to remove expired tokens — MongoDB handles it natively and efficiently.

4.4.3 Indexes

The following indexes exist across the three collections:

Collection	Index Field(s)	Index Type	Purpose
USERS	username	Unique Index	Prevents duplicate usernames. Created automatically by unique: true on the schema field.
USERS	email	Unique Index	Prevents duplicate email addresses. Created automatically by unique: true.
TODOS	_id	Default Index	MongoDB creates a default _id index on every collection. Used for findTodoByIdAndUser() lookups.
REFRESH_TOKENS	token	Unique Index	Prevents storing the same token string more than once.
REFRESH_TOKENS	expiresAt	TTL Index	expireAfterSeconds: 0 — MongoDB automatically deletes documents when expiresAt < current time.

Table 4.4-1: Index Table

4.4.4 ObjectId References (Manual References Pattern)

MongoDB does not enforce relational foreign key constraints natively. Instead, the Daily Planner uses the Manual References pattern — storing an ObjectId value in a field and using Mongoose's ref option to declare the target collection. Two references are used:

- **TODOS.user_id →USERS._id (ref: 'USERS'):** Every todo stores its owner's ObjectId. All todo service functions query using both the todo _id and the user_id together, ensuring ownership isolation at the database query level.

- **REFRESH_TOKENS.user_id → USERS._id (ref: 'USERS')**: The refresh token stores its owner's ObjectId, used to look up the user when issuing a new access token during the /auth/refresh flow.

4.4.5 Connection Management

The MongoDB connection is managed in db.config.ts. The connectDB() function calls mongoose.connect(config.MONGO_URI) and logs success or calls process.exit(1) on failure — preventing the application from running in a broken state with no database. The disconnectDB() function is called during graceful shutdown (triggered by SIGTERM or SIGINT signals in server.ts), ensuring the connection is cleanly closed before the process exits and no in-flight operations are abandoned.

4.4.6 Environment-Based Configuration

The MongoDB connection string is read from the MONGO_URI environment variable, loaded from a .env file at startup via dotenv. The env.config.ts file exports this as part of a typed config object. Using an environment variable rather than a hardcoded connection string means the same codebase can connect to a local MongoDB instance in development and a MongoDB Atlas cluster in production without any code changes — only the .env file differs between environments.

4.5 Data Models

4.5.1 User Model — USERS Collection

Field	Type	Required	Description
username	String	Yes	Unique user identifier. Trimmed. unique: true creates a MongoDB unique index.
password	String	Yes	bcrypt-hashed password string. select: false — excluded from all standard queries.
fullname	String	Yes	User's full display name. Trimmed.
email	String	Yes	Unique email address. Lowercase and trimmed automatically. unique: true
phone	String	No	expireAfterSeconds: 0 — Optional contact phone number.
gender	String(enum)	Yes	Enum: 'male', 'female', 'other'. Validated at the schema level.
avatar	String	No	Profile image URL. Auto-generated from DiceBear API on first save if absent.
createdAt	Date	Auto	Insertion timestamp, managed by Mongoose timestamps option.
updatedAt	Date	Auto	Last-modified timestamp, managed by Mongoose timestamps option.

Table 4.5-1: users collection

4.5.2 Todo Model — TODOS Collection

Field	Type	Required	Description
user_id	ObjectId(ref)	Yes	References the owner document in USERS. Enables populate() queries.
title	String	Yes	Title of the task. Trimmed. Required at both schema and controller level.
description	String	No	Optional longer description of the task. Trimmed.
completed	Boolean	No	Task completion flag. Defaults to false on creation.
start_date	Date	No	Task start date. Defaults to Date.now if not supplied
due_date	Date	Yes	Deadline for the task. Required at schema and controller level.
createdAt	Date	Auto	Insertion timestamp, managed by Mongoose timestamps option.
updatedAt	Date	Auto	Last-modified timestamp, updated automatically on every save().

Table 4.5-2: Todos collection

4.5.3 Refresh Token Model — REFRESH_TOKENS Collection

Field	Type	Required	Description
user_id	ObjectId(ref)	Yes	References the token owner in USERS. Used to fetch the user during token refresh.
token	String	Yes	Raw refresh token string. unique: true prevents duplicate token documents.
expiresAt	Date	Yes	Expiry datetime. MongoDB TTL index deletes the document automatically after this date.
createdAt	Date	Auto	Insertion timestamp, managed by Mongoose timestamps option.

Table 4.5-3: Refresh tokens collections

4.6 Authentication System

4.6.1 Registration (POST /auth/register)

signupPasswordMatcher() validates that both password and confirmPassword are present and equal. findUserByUsernameOrEmail() checks MongoDB for an existing document — if found, a 400 AppError is thrown. The password is hashed with bcrypt before the document is inserted. On success, an access token and a refresh token are generated and set as HTTP-only cookies. The response body returns the sanitized user (password removed) with HTTP 201.

4.6.2 Login (POST /auth/login)

Accepts an identifier field (username or email) queried with a MongoDB $or condition. The service explicitly adds .select('+password') to include the otherwise-excluded password hash. Both 'user not found' and 'wrong password' return the identical 'Invalid credentials.' error — intentionally preventing username enumeration. On success, tokens are issued identically to the registration flow.

4.6.3 Token Refresh (POST /auth/refresh)

Reads the refreshToken from the HTTP-only cookie — never from the request body. Queries REFRESH_TOKENS for a matching document. A secondary expiresAt < new Date() check handles the edge case where the MongoDB TTL monitor has not yet run. If valid, the associated user is fetched and a new access token is issued. The refresh token itself is not rotated in the current implementation.

4.6.4 Logout (POST /auth/logout)

Requires a valid access token. Calls deleteRefreshToken(req.userId!) to remove the token document from MongoDB, then clears both cookies. Any subsequent use of the old refresh token will find no matching document and receive a 403.

4.6.5 Current User (GET /auth/me)

Fetches the full user document by req.userId (injected by authMiddleware) and returns it sanitized. Useful for a frontend to recover session state on page load without using localStorage.

4.6.6 Cookie & JWT Security Configuration

- httpOnly: true — Cookies are inaccessible to JavaScript. Prevents XSS-based token theft.
- secure: true in production — Cookies are only transmitted over HTTPS.
- sameSite: 'strict' — Cookies are blocked on cross-site requests, providing CSRF protection.
- Access token maxAge: 15 minutes (900,000 ms). Refresh token maxAge: 7 days (604,800,000 ms).
- JWT secrets and expirations loaded from environment variables. jwt.util.ts throws explicit AppErrors if either is undefined, preventing tokens being signed with undefined secrets.

4.7 API Endpoints

4.7.1 Authentication Routes — Prefix: /auth

Method	Endpoint	Auth	Description
POST	/auth/register	No	Register a new user account.

POST	/auth/login	No	Log in with username/email and password.
POST	/auth/logout	Yes	Invalidate session and clear cookies.
POST	/auth/refresh	No	Refresh access token via refresh cookie.
GET	/auth/me	Yes	Return currently authenticated user profile.

Table 4.7-1: Authentication Routes

4.7.2 Todo Routes — Prefix: /todos (All Require Authentication)

Method	Endpoint	Auth	Description
GET	/todos	Yes	Retrieve all todos belonging to the current user
POST	/auth/login	Yes	Create a new todo item for the current user.
PUT	/auth/logout	Yes	Update any fields of a specific todo by ID.
DELETE	/auth/refresh	Yes	Permanently delete a specific todo by ID.

Table 4.7-2: Todo Routes

4.7.3 Request & Response Examples

POST /auth/register — Request Body

{

"fullname": "Kailash Badu",

"username": "kaibad",

"email": "kaibad@example.com",

"phone": "9800000000", "gender": "male",

"password": "Secret@123",

"confirmPassword": "Secret@123"

}

POST /auth/register — Success Response (201)

{

"success": true,

"message": "User Registered Successful.",

"user": { "_id": "664f1a...",

"username": "kaibad",

"fullname": "Kailash Badu",

"email": "kaibad@example.com",

"gender": "male",

"avatar": "https://api.dicebear.com/9.x/notionists/svg?seed=kaibad"

}

POST /todos — Request Body

{

"title": "Finish project report",

"description": "Complete all sections and export to docx",

"start_date": "2024-03-01",

"due_date": "2024-03-07"

}

POST /todos — Success Response (201)

{

"success": true,

"todo": { "_id": "665a2b...",

"user_id": "664f1a...",

"title": "Finish project report",

"completed": false,

"start_date": "2024-03-01T00:00:00.000Z",

"due_date": "2024-03-07T00:00:00.000Z",

"createdAt": "2024-03-01T10:00:00.000Z"

}

4.8 Middleware Pipeline & Error Handling

Every incoming request passes through the following middleware chain in order:

- **cookieParser():-** Parses the Cookie header and populates req.cookies.
- **express.json():-** Parses JSON request bodies into req.body.
- **express.urlencoded():-** Parses URL-encoded form data.
- **express.static():-** Serves static files from the public/ directory.
- **Route match:-** authMiddleware (where applicable), then the controller wrapped in catchAsync.
- **404 catch-all:-** Unmatched routes call next(new AppError('Route not found', 404)).
- **errorHandler:-** Registered last; formats all errors into a consistent JSON response.

4.8.1 catchAsync & AppError

Every async controller is wrapped with catchAsync before being registered, which attaches a .catch(next) to the returned Promise. Any thrown AppError or unexpected error is forwarded to the global errorHandler automatically. AppError extends Error with a statusCode and a status string derived from the code ('Client Error' for 4xx, 'Server Error' for 5xx). The prototype chain is explicitly fixed with Object.setPrototypeOf() to ensure instanceof AppError works correctly after TypeScript compilation.

4.8.2 Error Response Shape

{
"status": "Client Error",
"message": "Todo not found.",
"stack": "..."
}

Note: stack included only in development mode

5 CONCLUSION

The Daily Planner project delivers a secure, well-structured REST API backend demonstrating strong command of the TypeScript/Node.js/Express/MongoDB stack. The codebase is clean and

consistently typed, with every layer — configuration, data modelling, business logic, routing, and cross-cutting concerns — carrying a single, clear responsibility.

The MongoDB design is thoughtful: the USERS schema hides passwords by default using select: false; the TODOS schema uses timestamps and ObjectId references for ownership isolation; and the REFRESH_TOKENS schema leverages a native MongoDB TTL index to automate session cleanup without any scheduled jobs or manual intervention.

The authentication system reflects a strong grasp of web security: HTTP-only and SameSite cookies block XSS and CSRF attacks; the intentionally generic login error prevents username enumeration; and the dual-token architecture limits the blast radius of a compromised access token to its short 15-minute lifetime. The four UML diagrams — Use Case, Class, Sequence, and ER — together provide a complete picture of the system's actors, data structures, code organisation, and runtime behaviour.

The project successfully meets all its stated objectives and serves as a robust foundation for further feature development.

6 RECOMMENDATIONS

6.1 Input Validation with Zod

Manual field checks inside controllers (if (!title) throw...) should be replaced with a Zod schema at the route layer. Zod provides declarative, type-safe validation that generates structured error messages for every missing or malformed field, and its inferred TypeScript types eliminate redundant type annotations in controllers.

6.2 Refresh Token Rotation

getOrCreateRefreshToken() currently reuses an existing token rather than issuing a new one on every refresh call. Implementing rotation — deleting the old token and inserting a fresh one on each /auth/refresh request — would prevent replay attacks and allow detection of token theft (a reused rotated token signals a compromised session).

6.3 Rate Limiting on Auth Endpoints

The /auth/login and /auth/register endpoints are unprotected against brute-force and credential-stuffing. Adding express-rate-limit to these routes (e.g. 10 requests per 15 minutes per IP) would substantially raise the cost of automated attacks.

6.4 MongoDB Compound Index on TODOS

The current implementation queries todos using both user_id and _id. Adding a compound index { user_id: 1, _id: 1 } on the TODOS collection would allow MongoDB to satisfy these queries

entirely from the index without scanning the full collection, improving performance as the dataset grows.

6.5 Automated Testing with Jest and Supertest

No tests exist in the current codebase. Unit tests for utility functions and integration tests for API endpoints using Jest and Supertest would provide a regression safety net and enable confident refactoring.

6.6 Remove Default Secret Fallbacks

env.config.ts falls back to plain strings ('accesssecret', 'refreshsecret') if JWT secrets are absent from the environment. The config should throw a hard startup error when these critical values are missing, preventing accidental insecure deployments.

6.7 CORS Configuration

Before deploying alongside a frontend on a separate domain, the cors package should be configured with an explicit allowedOrigins list to prevent unauthorised origins from calling the API.

7 User Interface & Application Screenshots

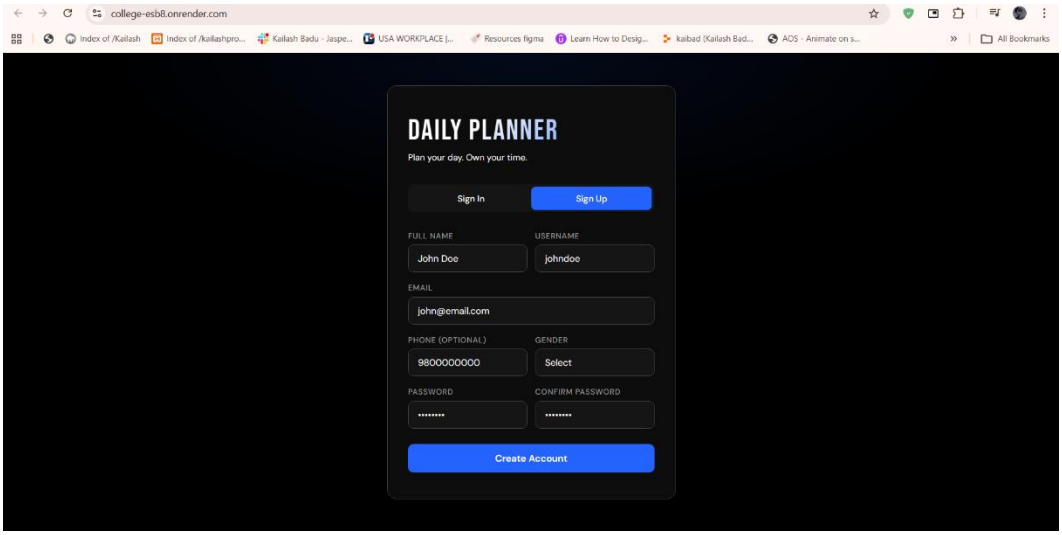

Figure 6.7.1 - Login Page

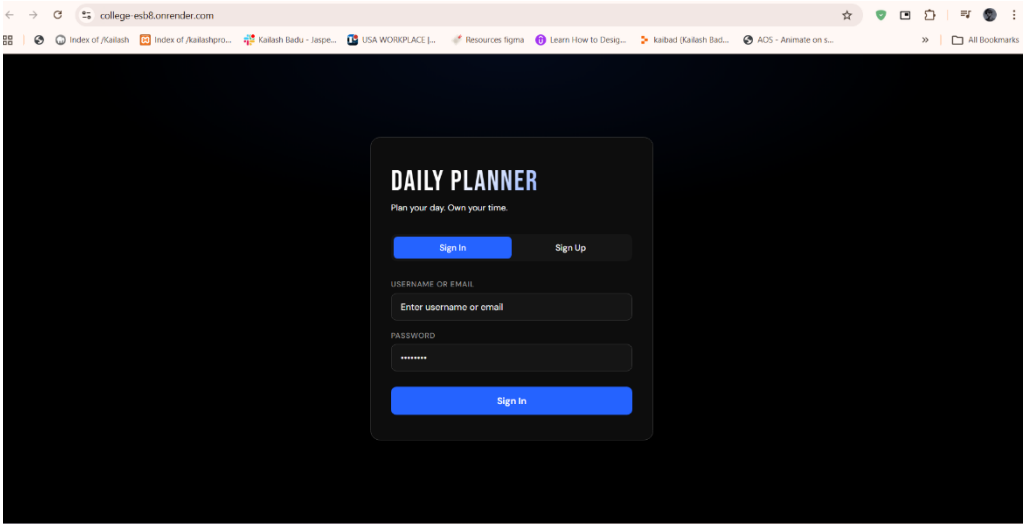

Figure 6.7.2 - Sign Up Page

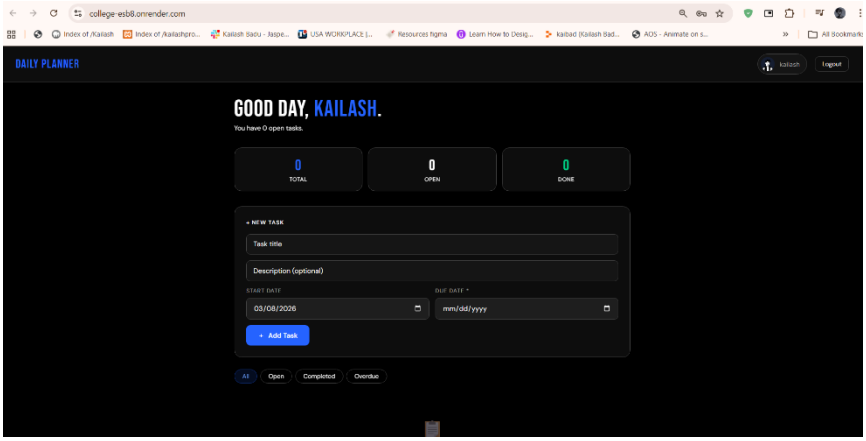

Figure 6.7.3 - Dashboard

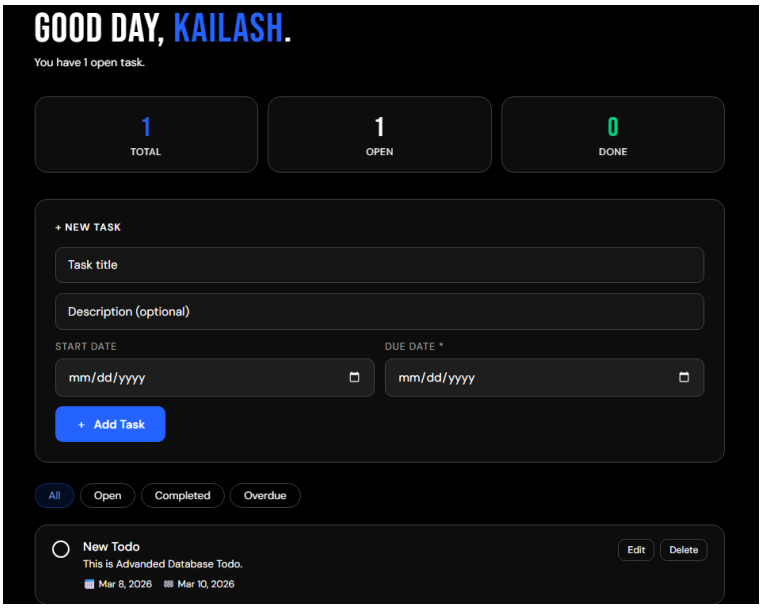

Figure 6.7.4 - Create and Get Todo

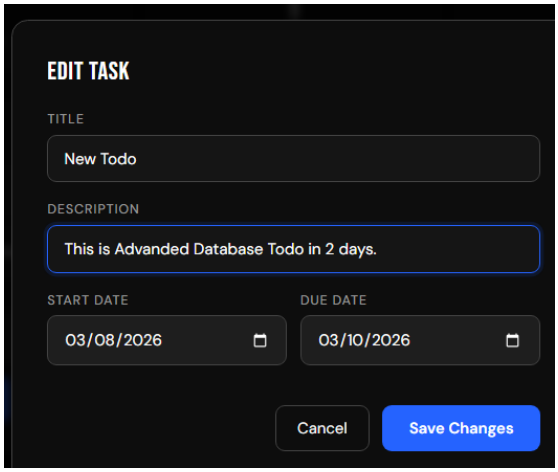

Figure 6.7.5 - Update Todo

8 References

[1] "MongoDB," 2025. [Online]. Available: https://www.mongodb.com/docs/manual/.

[2] "Mongoose," 2025. [Online]. Available: https://mongoosejs.com/docs/guide.html.

[3] "TTL Indexes," 2025. [Online]. Available: TTL Indexes — MongoDB Manual. https://www.mongodb.com/docs/manual/core/index-ttl/.

[4] "Node js," 2025. [Online]. Available: https://nodejs.org/docs/latest/api/.

[5] "Express js," 2025. [Online]. Available: https://expressjs.com/.

[6] "TypeScript Handbook," 2025. [Online]. Available: https://www.typescriptlang.org/docs/handbook/intro.html.

[7] "Introduction to JSON Web Tokens," 2025. [Online]. Available: https://www.jwt.io/introduction.

[8] "bcryptjs," 2025. [Online]. Available: https://www.npmjs.com/package/bcryptjs.

[9] "cookie-parser," 2024, [Online]. Available: https://www.npmjs.com/package/cookie-parser.

[10] "dotenv," 2026, [Online]. Available: https://github.com/motdotla/dotenv#readme.